Structured Streaming in Apache Spark

In this train, we'll explore how to setup and use Spark Structured Streaming to create, manage and optimize streaming pipelines in Apache Spark.

Learning objectives

By the end of this train you should be able to:

- [Placeholder for objective 1]
- [Placeholder for objective 2]
- [Placeholder for objective 3]

Learning Objective

- Be able to create a structured streaming pipeline and use Spark Structured Streaming. Integrate streaming into batch jobs into a lakehouse architecture.

1. Introduction to Structured Streaming

Structured Streaming is a scalable and fault-tolerant stream processing engine built on Spark SQL. It treats streaming data as an unbounded table, allowing batch-like operations with low latency.

Key Concepts:

- Micro-batch processing: Processes data in small batches (default mode).
- Continuous processing: Ultra-low latency (experimental).
- Event-time processing: Handles late data with watermarks.
- Exactly-once guarantees: Ensures no duplicate or missing records.

2. Setting Up a Structured Streaming Pipeline

2.1. Reading From a Streaming Source

Structured Streaming supports multiple sources:

- Kafka
- File-based sources (CSV, JSON, Parquet)
- Socket (for testing)

Example: Reading from a Socket (for Testing)

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("StructuredStreamingDemo") \
    .getOrCreate()
# Read from a socket (for testing)

lines = spark \
    .readStream \
    .format("socket") \
    .option("host", "localhost") \
    .option("port", 9999) \
    .load()
```

2.2. Processing the Stream

```
from pyspark.sql.functions import split, col
# Split lines into words

words = lines.select(split(col("value"), " ").alias("word"))
# Count word occurrences

word_counts = words.groupBy("word").count()
```

2.3. Writing to a Sink

Output modes:

- Append (only new rows)

- Complete (all rows)
- Update (only changed rows)

Example: Writing to Console

```
query = word_counts \
    .writeStream \
    .outputMode("complete") \
    .format("console") \
    .start()

query.awaitTermination()  # Keep the query running
```

3. Integrating Streaming With Batch in a Lakehouse

A Lakehouse combines streaming + batch processing on a single data platform (Delta Lake, Iceberg, Hudi).

3.1. Writing to Delta Lake

```
output_path = "s3a://my-bucket/streaming_output/"

query = word_counts \
    .writeStream \
    .outputMode("complete") \
    .format("delta") \
    .option("checkpointLocation", f"{output_path}/checkpoint") \
    .start(output_path)
```

3.2. Reading Streaming & Batch Data Together

```
# Read streaming data

stream_df = spark.readStream.format("delta").load(output_path)
# Read batch data

batch_df = spark.read.format("delta").load("s3a://my-bucket/batch_data/")
# Union both

combined_df = stream_df.union(batch_df)
```

4. Optimizing Structured Streaming

4.1. Trigger Intervals

Control batch frequency:

```
query = word_counts \
    .writeStream \
    .trigger(processingTime="10 seconds") \
    .start()
```

4.2. Watermarking for Late Data

```
watermarked_df = words \
    .withWatermark("timestamp", "10 minutes")  # Discard data >10 mins late
```

4.3. Checkpointing for Fault Tolerance

```
query = word_counts \
    .writeStream \
    .option("checkpointLocation", "/checkpoint/dir") \
    .start()
```

Conclusion

Structured Streaming in Apache Spark provides a powerful, unified API for processing real-time and batch data seamlessly. By leveraging micro-batch or continuous processing, it ensures fault tolerance, exactly-once semantics, and easy integration with Lakehouse architectures. With this knowledge, you're now equipped to build scalable streaming pipelines that bridge real-time and historical analytics efficiently.

▼Key Takeaways ✖

- Structured Streaming treats streams as unbounded tables.
- Supports exactly-once processing and event-time handling.
- Integrates seamlessly with batch jobs in Lakehouse architectures.
- Optimize with triggers, watermarks, and checkpoints.

▼Key Takeaways ✖

- **Takeaway 1:** [Placeholder for key takeaway 1]
- **Takeaway 2:** [Placeholder for key takeaway 2]
- **Takeaway 3:** [Placeholder for key takeaway 3]